

5.1 System Requirements Analysis

5.1.1 System boundary, actors and use cases

Figure 70 shows the boundaries of the proposed system, the actor on the system and the key use cases. The system boundary includes the images or video uploaded by the user and the AI modules that provide analytics to the user. The key actor on the system is a farm owner who may be physically located on the farm or remote from the farm. As shown by the figure, the farm owner (actor) interacts with the system by first selecting a module from five options: daytime object detection, daytime change detection, daytime object tracking, night time object detection or night time object tracking. Based on their selection they will be prompted by the system to upload a single image, 2 images, or a video. The uploaded data will pass through the pre-processing module and the processed image(s) will be passed to the appropriate AI module based on the user selection. There is interaction between the different AI modules, for example if the selects daytime change detection then each user uploaded image will pass through the daytime object detection module to generate the detection results and the changes between the detected objects in each image will be presented to the user. The object tracking module also uses the results of the appropriate detection module to track objects in an uploaded video. If the user has selected daytime object tracking then the trained daytime object detector will be used to track objects in the uploaded video whereas if the user has selected the night time object tracking option then the trained night time detection model will be used to track objects in uploaded thermal video data. The module selected by the user allows them to achieve their desired goal or use case. As demonstrated by Figure 70, the system has the following use cases:

1. Detect intruders at night

The farm owner can select the nighttime detection module, upload their thermal imagery or video and detect intruders on their farm.

2. Find missing livestock

The farm owner can select the daytime detection module, upload daytime imagery or video and generate counts of livestock in different areas of their farm. They can use this data to identify missing livestock and keep track of the quantity of livestock in different areas of the farm.

3. Analyze farmland changes

The farm owner can select the change detection module and upload a video or several images to generate a change detection report summarizing how the quantity and location of objects on their farm has changed over time. They can use this to monitor the development of their farm.

4. Monitor farm remotely

The farm owner can select the object tracking module to monitor the movement of mobile objects on their farm. This module enables the farm owner to monitor the movement of objects on their farm while not being physically present at the farm.

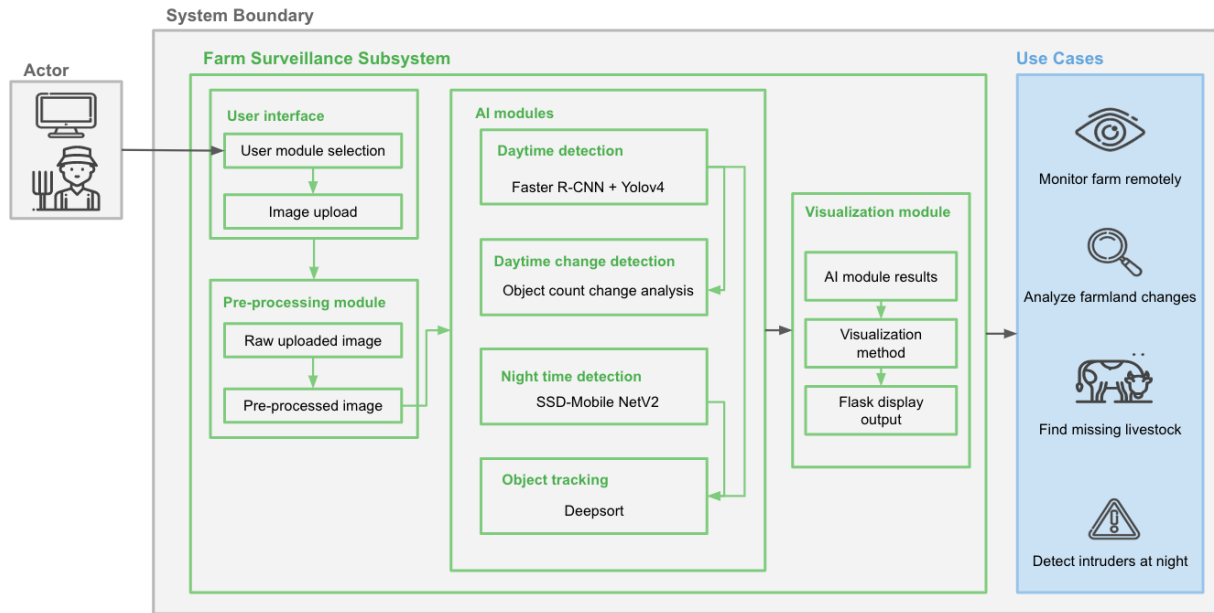


Figure 70. A diagram showing the system boundary, farm surveillance subsystem, use cases and actor on the system.

5.1.2 Data analytics and Machine Learning capabilities

The high-level data analytics and machine learning capabilities that the system will provide are as follows:

1. Detect and count buildings, trees, livestock and vehicles from aerial imagery or videos taken during the day.

The capability to detect and count farm objects during the day is delivered by the daytime object detection module. The output of this module are bounding boxes and percentages indicating the confidence in the prediction as well as a summary of the detected objects and their counts.

2. Detect humans and animals from infrared imagery or video.

The nighttime detection module provides the capability to detect humans and animals from infrared imagery or video taken at night. The output is a bounding box surrounding the detected human or animal.

3. Track mobile farm objects during the day.

The tracking module provides the capability of tracking farm objects in aerial videos taken over farmland. The module uses the object detection output to track the movement of objects in videos. The output of this module is bounding boxes and labels added to video frames that track the movement of the object throughout the video.

4. Detect farmland changes

The change detection module uses the output of the object detection modules to identify changes in the location and quantity of objects based on the IoU between detected bounding boxes. The output of this module is a text summary of all detected changes in object location and quantity.

5.2 System Design

5.2.1 System Architecture and Infrastructure

Figure 71 shows the frontend and backend system architecture of the smart farmland surveillance system. For the frontend part, the user (usually a farm owner) can upload the image or video they want to detect something in and the email address to receive the summary/alert. A user can upload the data and then choose the target object they want to detect: livestock, building, tree, vehicle, or all of the above. Alternatively, users can enter the address they want to check directly and our system will automatically download the satellite image from the Google Maps API. Since the resolution of images is limited by Google Map, this method could only detect relatively large objects like buildings and vehicles. The backend system is divided into three parts: the data preprocess module, detection, and storage. The storage is configured so that all the uploaded data will be saved in the cloud for 7 days. After uploading, the data will go to the preprocess module and then go through different detection modules according to the choices made by the user. For example, if the user wants to know the changes of all objects on the farmland, the

images will go to the daytime detection module first, the YOLOv4/Faster R-CNN ResNet50 will detect all the objects in the old and new images. Then the results will be sent to the change detection module and any changes will be detected. If the user wants to detect the intruder at night, the preprocessed data will go to the nighttime detection module, the YOLOv4/SSD-MobileNetV2 will detect if there's an intruder or not. Once all the detections are completed, the detected image(s) with bounding boxes and labels with a summary will be sent back to the user on the screen to report the information of objects on the farmland. If there are intruders detected or changes found, an alert will show on the screen with an email sent to the users.

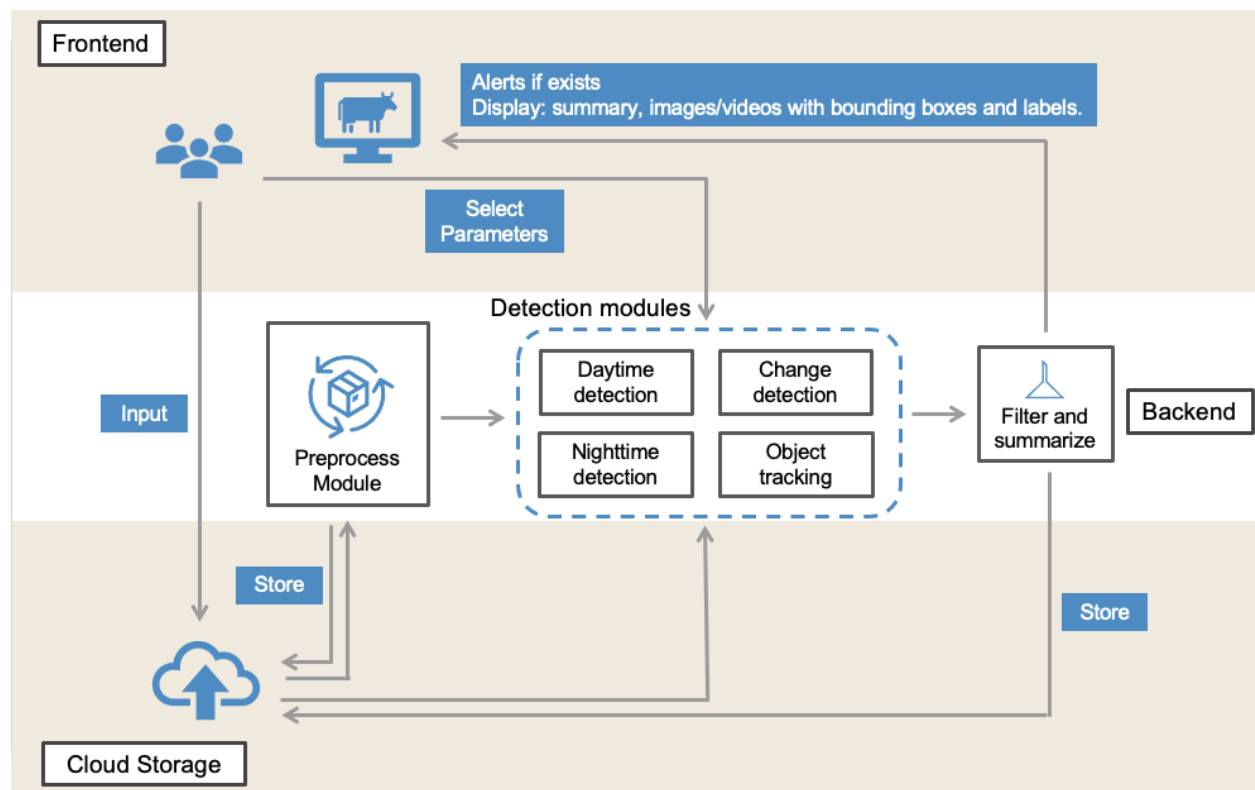


Figure 71. System Architecture

5.2.2 System Supporting Platforms and Cloud Environment

For this project, we propose to develop a smart farm surveillance system and its supporting frameworks with integrated service tools based on existing open-source tools. For the backend, model development is performed with Darknet framework, Tensorflow and OpenCV deep learning packages. For the frontend, the application will be developed by Flask micro web framework. The integration of four models, as well as training data image processing, are organized in Google Colab. With respect to the training data storage, all data is saved both locally and in Google Cloud. With the integration of Colab and Google cloud, large datasets can be retrieved and trained for complex improved deep learning models. Besides the training data, the testing and user data will

also be saved in the cloud for easier access and management. Detailed data flow is discussed in the next part.

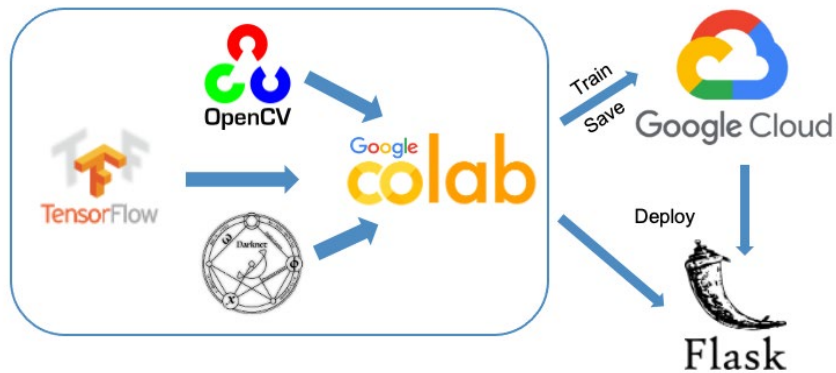


Figure 72. Supporting platforms, frameworks and a cloud environment.

5.2.3 System Data Management Solution

Figure 73 shows the data flow of our system. First of all, users need to choose the module they want to use and the objects they want to detect. If they want to receive the alert, they can also enter their email addresses. Since we don't require users to register for the application, the email addresses will only be saved as a temporary variable, the location addresses entered by the user are saved in the same way. Then the users can upload their images/videos to the cloud. Our preprocessing module will do sharpening and normalization on the data and save them on the cloud. Then the detection modules will detect the data and output the detection results and images/videos with bounding boxes and labels to the cloud. In the end, the results will be summarized and displayed on the monitor. Besides, all this uploaded and generated data will be deleted after the user closes the web page to save storage space and protect the user's privacy.

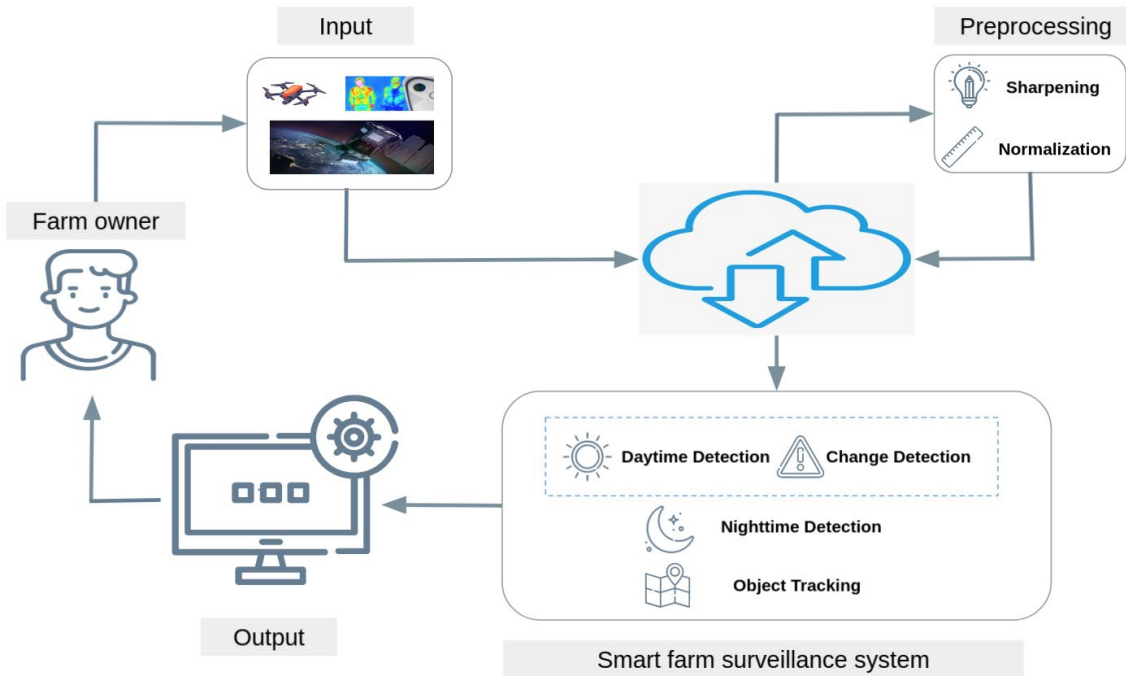


Figure 73. Data flow chart from input to output

5.2.4 System User Interface

Figure 74 shows examples of the interface for the smart farm surveillance system. The left image shows users can choose the scenarios and objects by themselves on the left sidebar and then upload the images/videos on the center of the right side. After the data is successfully uploaded, the user can click the start button to get the results. The right image displays the result page, users can browse the detection result image/videos with bounding boxes and labels, alert if there exists, and farmland summary in turn from the top to the bottom. As shown in the figure, the background color of the alert is red to attract more attention.



Figure 74. Examples of the interface for the smart farm surveillance system

5.3 System Development

5.3.1 AI and Machine Learning Models Development

The main goal for this project is to allow the user (farm owner) to utilize the power of AI to quickly survey their farmland. The system is designed to have four different use cases – nighttime intruder detection, farmland changes analysis, livestock surveillance, and on demand remote monitoring (drone video surveillance) as mentioned previously in section 5.1. To maximize the accuracy of our model, we fine-tuned the livestock detection YOLOv4 model and created an ensemble model through integrating YOLOv4 model and Faster R-CNN model to detect tree, building and vehicle. Therefore, we will implement both YOLO and Faster R-CNN together to take care of our object detection task and YOLO + DeepSort for object tracking (video surveillance). Below is the breakdown of the models used in the system with their corresponding functions:

- I. Ensemble model (YOLOv4 + Faster R-CNN) – Building, tree, livestock, vehicle detection
- II. SSD MobileNetV2 – Night object detection
- III. DeepSort + YOLO v4 – Remote monitoring
- IV. Change of the number of detected objects for each class – Change detection

To ensure consistency between models, both object detection (daytime) and object tracking are using the same dataset – self collected HRO and GEP images for training, while the night time detection is on a separate dataset for it requires infrared thermal images. To improve the training accuracy, all detection models are trained separately for a specific object. Once the training weight for each object is ready, the model weights are ensembled together using Weighted Boxes Fusion into a single predictive model. Model accuracy will be accessed using the mean Average Precision (mAP), Intersection over Union (IoU), and precision-recall curve.

The integrated predictive model development can be classified into 3 stages – initial, intermediate, and final.

i. Initial stage – Data collection and image annotation

For there is no existing dataset that can satisfy our requirement for model training, we decided to collect our own farmland remote sensing imagery dataset. A high spatial resolution image is required for object detection, therefore, we have utilized USGS EarthExplorer to collect HRO images and Google Earth Pro to collect GEP images. The images collected covered different farmlands located across the US. As the background of the images, for example trees, may appear differently during time of the year, images for the same location are collected at different time frames to reflect seasonal changes. The collected images will be annotated with labels and bounding boxes using Labellmg.

ii. Intermediate stage – Model training and testing

Once the self-collected dataset is built, several models are trained on it for object detection.

- Daytime object detection – An ensemble model based on the YOLOv4 and Faster R-CNN detection results is used to detect objects in daytime imagery. The Faster R-CNN model consists of three stages, (i) feature extraction from the input image, (ii) RPN and RoI pooling, that is the feature map extracted from stage one is passed to RPN where the region of interest is identified and the pooling layer will bring down the identified features to a fixed number, and (iii) classification, this is the output consists of 2 fully connected layers: a softmax classifier which outputs the class probability and a regression CNN which outputs the bounding box predictions. The YOLOv4 model is based on convolution neural network (CNN) and it is optimized for parallel computations, which is why the YOLO family is known for fast prediction. The model architecture of YOLOv4 consists of (i) backbone, which is CSPDarknet53, (ii) neck, which is the spatial pyramid pooling additional module, PANet path-aggregation, and (iii) head, which is YOLOv3. YOLOv4 uses the “Bag of Freebies” (BoF) and “Bag of Specials” (BoS) object detection procedures. The BoF deploys CutMix and Mosaic data augmentation, DropBlock regularization, and class label smoothing, while BoS deploy Mish activation, Cross-stage partial connections (CSP), and Multi-input weighted residual connections (MiWRC) for the backbone. The training process needs more time than Faster R-CNN. Based on the training session of our model, for a 6000 images dataset, YOLOv4 requires more than 20 hours and Faster R-CNN trained in less than an hour.

- Nighttime object detection – SSD uses many of the same ideas as YOLO but works even better—the main difference is that YOLO makes predictions for only a single feature map while SSD combines predictions across multiple feature maps at different sizes [97]. MobileNetV2 is a lightweight, small, and fast network while maintaining an acceptable standard of accuracy, which is specifically tailored for mobile and resource constrained environments [91, 93]. In the case of object detection with SSD, we want to know not just these high-level features but also lower-level ones. Since object detection is more complicated than classification, SSD adds many additional convolutional layers on top of the base network. So it's important to have a feature extractor like MobileNetV2 that is fast. SSD-MobileNetV2 performs well on large objects but poorly for small objects. For satellite images, it's not a good choice, but for night intruder detection, especially when the user wants to detect on a mobile device, it's a good choice.

- Remote monitoring – DeepSort + YOLO v4 is used to track and detect objects in drone video for real time remote monitoring. The model consists of two steps – the first step is to use YOLO v4 for object detection and the second step is multiple object tracking using DeepSort. Details on YOLO v4 are explained in daytime object detection. Tracking multiple objects at the same time is a data correlation problem. DeepSort uses Kalman filtering to process the correlation of frame-by-frame data, Hungarian algorithm for correlation measurement, and CNN network for training and deep feature extraction on each frame to solve occlusion problems.

- Change detection – Once the object detection models are built, the change detection is simply calculating the counting number difference between two time series images. The two input images will first pass through the detection module to output the number of objects detected in each class, then the number difference between each class will be calculated to check for changes.

iii. Final stage – Model integration and deployment.

All the developed AI and machine learning modules will be integrated together into a single system, deployed with flask, and a graphical user interface (GUI) will allow the end user to interact with the system.

5.3.2 Implement Designed System

The final designed system can be classified into four functional components and each component will handle one use case. As mentioned in 5.1, our model is composed of four different use cases – object detection (building, tree, vehicle, livestock), night intruder detection (infrared image), real-time drone remote monitoring (multiple object tracking and detection), and change detection, for example missing livestock. A GUI will be designed and implemented with the final system to allow the user to interact and select their desired use cases. The system is required to identify five different features: building, livestock, vehicle, tree, and human (night intruder). Each feature is classified and handled by different models and the results from each model are combined together to generate one single output prediction image with all the detected objects.

For daytime detection, the user can select their location of interest or upload an image and the input images are preprocessed at the backend and fed into the detection module to process and output an image with bounding boxes and labels of detected objects (tree, building, vehicle, and livestock). For night intruder detection, input images are captured by infrared camera located on the farmland, preprocessed and fed into the detection module to output prediction. For change detection, users can select this option from GUI and upload two time series images to track for changes. For remote monitoring, the user can deploy a drone to fly over the farmland to capture video. The input videos are fed into the tracking module to output detected objects in the video.

5.3.3 Input and Output Requirements, Supporting Systems and Solution APIs

- Input datasets: Depending on the use cases, the input dataset will come from different sources. For daytime detection, the user can select their location of interest from GUI and the image will be downloaded from the web using the Google Map API. For change detection, the input dataset comes from the user (user will upload their own images). For night intruder detection, the dataset comes from an infrared camera installed in the farmland and lastly, the remote monitoring dataset are videos captured by drone. All input datasets are stored in the cloud and passed through the corresponding module for classification based on the user selected use case.
- Expected outputs: For detection and tracking models, a single image (or video for tracking) is generated with labels and bounding boxes, which is a combination result from each individual feature detection model. For change detection, the system will output a warning statement if a change is detected and a visual report will be generated summarizing the changes (figure 84).
- Supporting system: Infrared camera is used in our system to capture night time images and all models are trained on Google Colab with GPU.

- Solution APIs: In order to allow a user to select their location of interest to detect the objects, the Google Map API is integrated in our system. This allows the user to select the location from a drop down menu or manually enter the address of interest, then the corresponding image will be pulled from the internet. Tensorflow object detection API and COCO API are used by Faster R-CNN to perform detection and evaluation.

5.4 System Support Environment

Several environments are used for this project to support AI-functional and storage mechanisms, as well as deployment. The models mostly run on Google Colab to utilize the GPU powered provided by Colab, Google Drive is the main storage for the dataset, Github is used as our code repository, and Flask is used for deployment.

Google Colab and Google Drive

Due to the limitations of local computing power, we trained our model primarily with Google Colab. The use of GPU greatly accelerates the training speed of our object detection models. At the same time, all the training, validation, and testing data is stored in Google Drive. By combining Colab and Drive, we were able to collaborate more efficiently and ensemble our models more easily.

Github

One cohesive GitHub repository is set up for our system to record all the project's models' weights, module scripts, and part of the test instances. Versions are stored for each update, so we can revert to an earlier version. It also logs summaries of what was changed so we can better track progress.

Flask

Flask is a micro web framework written in Python. It can help end-users interact with our models directly from their web browser without needing any libraries, code files, etc. Flask enables us to create web applications very easily, hence enabling us to focus more on other important parts. In short, Flask has the following advantages: easy to use, built-in development server and debugger, integrated unit testing support, RESTful request dispatching, extensively documented.

As per the requirements, Flask itself supports Python 3.6 and newer. Our project needs Tensorflow 2.x, Keras 2.3.1, ImageZMQ, pillow, opencv-python 4.2.0 and the application will be tested locally on a MacBook Pro with 32GB memory first.